LLM エージェントの履歴要約と終了制御による実行費用の評価

桐生　篤希[†a)]（学生員）
菊澤　百々菜[†]（学生員）
木村　秀明[†]（正員：フェロー）

Evaluating Execution Cost of History Summarization and Termination Control for LLM Agents

Atsuki KIRYU[†a)], Momona KIKUZAWA[†], Student Member, Hideaki KIMURA[†], Fellow Member

† 中部大学大学院工学研究科情報工学専攻，愛知県
Department of Computer Science, Graduate School of Engineering, Chubu University, 1200 Matsumoto-cho, Aichi, 487-0027 Japan
a) E-mail: tp26013-0852@sti.chubu.ac.jp

あらまし　LLM エージェントの履歴要約方式と API 実行記録に基づく終了制御を独立に評価する．要約生成を含む全モデル呼出しを計測し，今回の既知課題で成功を保ちながら総トークン数が減少したことを確認した．呼出し配置と完了後の生成に着目した比較結果を示す．

キーワード　LLM エージェント，ハーネス，履歴要約，終了制御，トークン使用量

1. はじめに

日常業務のデータ整理や事務処理を支援する大規模言語モデル（LLM）の研究が進んでいる．写真整理や精算では，文章生成に加え，外部アプリケーションで対象の検索，状態の変更，結果の確認を連続して行う必要がある．推論と行動を繰り返す ReAct や，コードを行動として実行する CodeAct は，こうした処理を支える[1]，[2]．

しかし，実行が長期化すると，過去の操作や観測結果を次の生成へ繰り返し入力する費用が増大する．また，作業が完了した後もエージェントが確認や生成を続ければ，成果を増やさずに計算資源を消費し得る．単純に履歴を短縮するだけでは課題の遂行に必要な情報を失うおそれがあり，一方で早期に終了させれば課題が未完了に終わり得る．そのため，課題の最終的な成功と実行全体の費用を同時に検証する必要がある．

そこで本稿では，モデルの再学習を伴わずに実行を管理するハーネスを用い，履歴要約の呼出し配置と終了制御を評価する．履歴要約では，要約を生成する時点，行動生成と要約の呼出しを分けるかどうか，要約機会の上限を比較軸とする．また，終了制御では，実行中に取得できる Application Programming Interface（API）の要求と応答を記録し，この API 実行記録に基づく制御を通常継続と比較する．両者を独立した実験として実施し，要約生成を含む全モデル呼出しの総トークン数と，課題の最終的な成功を調べる．

2. 関連研究

長文中の情報の位置は性能に影響する[3]．入力圧縮では，LLMLingua は予算制御とトークン選別を，LLMLingua-2 は蒸留データによるトークン分類を，LongLLMLingua は長文の重要情報を残す圧縮を扱う[4]，[5]，[6]．

長期実行では，ACON は観測・履歴の圧縮方針を，TokenPilot は入力圧縮とキャッシュ継続性を検討している[7]，[8]．Generative Agents は経験の記憶・抽象化・検索を，Reflexion は言語的フィードバックの後続試行への利用を扱う[9]，[10]．そこで，要約の追加費用も考慮した方式選択を課題とし，要約機会の上限と行動生成との呼出し配置を比較軸に，成功と総トークン数を評価する．

終了制御では，Runaway が終了指示と外部の完了検証を検討している[11]．本研究では，既知の要求形式に対する完了条件を API 実行記録から検査し，次の生成の要否をハーネスで判断する．通常継続と比較し，成功，API 要求数，総トークン数への影響を評価する．

3. 提案手法

ハーネスとは，モデルの外側で入力と実行の継続を管理するプログラムを指す．本稿では，既存の要約を用いて入力履歴を更新する処理と，API 実行記録に基づいて次の生成の要否を判断する処理をそれぞれ構成する．図 1 の実験 A に要約と行動生成の呼出し配置を，実験 B に完了判定に基づく終了制御を示す．両者は個別に評価し，組合せによる削減効果は本稿の対象外とする．

専用 1 回方式と専用反復方式では，行動と要約の呼出しを分ける構成を共通とし，要約機会の上限を比較する．専用反復方式と行動併記方式では，機会の上限を揃え，要約候補を専用呼出しで生成するか，行動と同じ応答に含めるかを比較する．候補の採否と入力の更新は，いずれもモデルの外側のハーネスが担う．

まず，課題と履歴からモデルが行動を生成し，ハーネスがプログラムを実行して行動と結果を記録する．本稿では，行動生成から実行結果の記録までを 1 ターンと定義する．入力が短くても追加処理で総量が増え得るため，要約生成を含む実行全体のモデル呼出しを集計し，課題の最終成功と比較する．

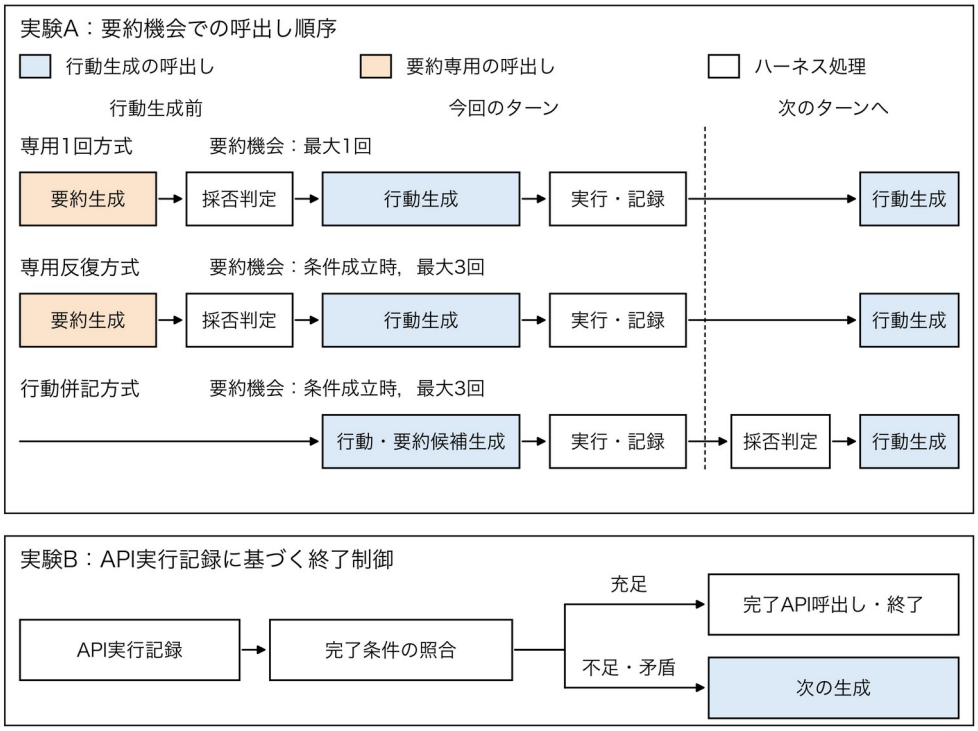


図1　履歴要約方式と終了制御

3.1　入力履歴を短くする制御

保存した全履歴を残し，モデルへ渡す古い履歴を要約に置き換える．採否判定前の要約を要約候補，要約生成だけのモデル呼出しを専用呼出しと呼ぶ．

専用1回方式は，入力長が閾値に達し，要約可能な古い履歴がある最初の機会に専用呼出しで候補を作り，採否を決めて次の行動生成へ進む．要約機会は最大1回とする．

専用反復方式は，初回は専用1回方式と同じで，以降は入力長と新たに完了したターン数の条件を満たすたび，採用済みの要約と未要約の古い履歴から候補を作る．最大3回の要約機会を設け，専用呼出し後に採否を決めて次の行動生成へ進む．

行動併記方式は，行動と要約候補を同じモデル応答に出力し，専用呼出しなしで最大3回の要約機会を設ける．応答内の全プログラムを実行・記録してから，次の生成前に候補の採否を決める．その実行結果を候補には含めず，行動と結果を次回入力に原文で残す．

要約は，完了した作業，次の操作に使う情報，残作業，不確かな点の4欄からなる．ハーネスは形式と文字数を検査し，元の指示・課題，要約，未要約の履歴を含む次回入力全体のトークン数が，不採用時より厳密に少なければ採用する．元の指示・課題と候補生成時の直近2ターンは原文で保持する．検査は構造と入力長に限り，要約の正確性は保証しない．

各欄には実行済みの作業，識別子・APIの呼出し方・保持した変数，未実施の操作，未確認事項を分けて残し，APIの正常応答だけで完了と推定しないよう指示する．

候補生成を試みる時点を要約機会と呼ぶ．不採用時は既存の要約と未要約の履歴を維持し，初回採用前は元の履歴を使う．不採用も1回の要約機会に数え，その場で再生成しない．専用1回方式でも生成・採用が0回の場合がある．閾値は4.1に示す．

候補の採用で入力を更新した回数を採用回数と呼び，要約機会数，モデル呼出し数と区別する．採用後の行動と結果も，次の要約機会まで未要約の履歴として入力に残す．

3.2　作業完了後の生成を省く制御

終了制御は，API実行記録から完了条件を調べる完了判定で，作業後の生成を省く．写真を作成時期別の指定先へ移す既知の1つの要求形式に，手書き規則を

用いる．応答内の全プログラムの実行・記録後，移動前の一覧・作成日時と API から得た現在年から，全対象の移動先と元の名前の保持を照合する．根拠が初めて揃えば完了 API を 1 回呼び，次の生成を省く．

移動前の対象集合の確定により，一部の移動だけで完了と判断することを防ぐ．根拠はその時点の API 実行記録に限り，モデルの完了宣言・推測，計算機の時刻，公式評価器の正解を用いない．

未対応の要求形式や記録の不足・矛盾があれば生成を続ける．通常継続は，この判定による終了を行わない．

3.3 総トークン数と課題の成功

条件 p のモデル呼出し集合を C_p とし，呼出し c の入力・出力トークン数をそれぞれ I_c，O_c とする．総トークン数 T_p を式(1)で求める．

$$T_p = \sum_{c \in C_p} \left(I_c + O_c \right) \tag{1}$$

行動・要約生成を対象とし，不採用候補や失敗時の既知の使用量も計上する．不明分は 0 とせず，既知分の小計と不明件数を記録する．

成功は実行後の最終状態を AppWorld の公式評価器で判定する[12]．正解や詳細を実行中のモデルへ返さず，成功数と同じ課題の成功維持を確認する．実験 A は課題別の総トークン数も示し，合計だけで増減を見落とさないようにする．

キャッシュ入力を含む論理トークン数は，課金額や実行時間を直接表さない[8]．要約生成を含む評価は ACON でも用いられる[7]．実験 A と B は独立に集計する．

4. 実験 A：要約の呼出し配置

4.1 条件

既知の写真操作，精算，返金，CSV 処理に 3 方式を各 1 回適用し，計 12 実行とした．MiniMax-M3 の GGUF IQ4_XS 量子化モデルを llama.cpp b11669 で実行し，文脈長は 65,536 トークンとした．乱数初期値 seed はモデル生成 57，環境 42 を基点とし，生成側は呼出しの役割と順序で増分した．

要約機会は入力長が 16,384 トークン以上で設ける．閾値は文脈長の 4 分の 1 とし，直近 2 ターンを保持する．専用反復・行動併記方式は，採否によらず前回の機会終了後に新たに 3 ターン以上の完了を要求する．初回は課題開始を起点とする．要約は 8,192 文字以下，専用呼出しの出力は 4,096 トークン以下とし，3.1 に従い採否を決める．要約機会の上限は実行を有限に保つために定めた．

各実行はモデル呼出し 80 回，総トークン数 450,000，1,800 秒，API 要求は写真 160 回，他課題 80 回を上限とした．温度は行動生成 1.0，専用呼出し 0.7，並列実行数 1，KV キャッシュ q8_0 とし，Apple M3 Ultra，メモリ 256 GiB で実行した．これらと保持ターン数，要約間隔・文字数・出力長，seed は開発比較の固定設定を継承した．写真の API 上限は，先行診断で 80 要求では未完了となり 160 要求へ拡張した値である．

4.2 全 4 課題の結果

表 1 に 4 課題の結果を示す．全 12 実行で課題の成功を確認し，使用量の欠測はなかった．

表 1　実験 A の課題別総トークン数

課題	専用 1 回方式	専用反復方式	行動併記方式
写真操作	139,871	139,871	129,374
精算	211,110	211,110	211,110
返金	102,087	102,087	102,087
CSV 処理	440,461	332,588	394,351
合計	893,529	785,656	836,922

4 課題合計は専用反復方式が最少で，専用 1 回方式より 107,873 トークン，12.07%少なかった．差は CSV 処理だけで生じ，他 3 課題は同値だった．

写真は全方式で要約を 1 回採用し，行動併記方式が他 2 方式より 10,497 トークン少なかった．精算と返金は全方式で総量が同じで，採用は 0 回だった．CSV 処理の採用は専用 1 回方式が 1 回，専用反復・行動併記方式が各 2 回だった．

行動併記方式の合計は専用 1 回方式より 6.34%少なく，専用反復方式より 51,266 トークン，6.53%多かった．後者との比較では，CSV 処理の 61,763 トークン増が写真での減少を上回った．

5. 実験 B：API 実行記録に基づく終了制御

5.1 API アクセス条件と比較手順

同じ写真課題に，API 文書取得が必須の文書必須条件と，公開 API 名からアクセスできる名前参照条件を設けた．各条件で通常継続と終了制御を各 1 回比較し，後者は保存済みの通常継続と初期状態，モデル，seed，指示・ツール説明，予算を揃えた．

モデルと実行基盤は 4.1 と同じで，seed の基点を 42 とした．実験 A とは履歴・設定を分け，上限はモデル

呼出し 80 回，450,000 トークン，1,800 秒，API160 要求とした．

3.2 の判定に追加のモデル呼出しは用いない．判定・保存・完了通知を含む実行時間を記録し，完了通知を API 要求数に含めた．

終了前の生成差を調べるため，終了制御側のモデルへの要求と応答の全文が通常継続側の先頭部分と完全に一致することを事前基準とし，API 操作記録と最終状態も照合した．

5.2　成功を保った総トークン数の削減

表 2 の全 4 実行が成功し，欠測はなかった．終了制御を用いた実行の総トークン数は通常継続より，文書必須で 30.15%，名前参照で 26.00%少なかった．完了通知を含めても，移動後の確認 API4 回とそれに伴うモデル呼出し 2 回が省かれ，API 要求も 4 回減った．

表 2　実験 B の総トークン数と呼出し回数

API 条件	終了方法	総トークン数	モデル呼出し数	API 要求数
文書必須	通常継続	130,439	15	104
文書必須	終了制御	91,116	13	100
名前参照	通常継続	151,664	14	103
名前参照	終了制御	112,233	12	99

API 操作などの前半記録と終了時の 13 個のデータベースのハッシュ値は通常継続と一致し，最終状態と成功を保った削減を確認した．

6. 考察

実験 A の 4 課題合計では，要約専用呼出しを省いても総量は最少とならず，全呼出しを計測する必要性を示した．実験 B は，API 実行記録から作業後の確認と生成を省く管理の有用性を示唆した．

本評価は既知課題，単一モデル・量子化設定，少数の seed・試行の開発実験の事後整理である．実験 B は各 1 回で全文一致基準を満たさず，生成過程の差も含み得る．平均的な因果効果，汎用的な終了保証，未見課題の品質維持，統計的非劣性の検証を今後の課題とする．

7. むすび

独立の 2 実験で，既知課題の成功を保った総トークン数の削減を観測した．今後は課題と要求形式を広げ，複数の seed で評価し，多様な作業の実行管理技術へ発展させていく予定である．

文　献

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” Proc. ICLR, May 2023.

[2] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng, and H. Ji, “Executable code actions elicit better LLM agents,” Proc. ICML, PMLR, vol.235, pp.50208–50232, July 2024.

[3] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” Trans. ACL, vol.12, pp.157–173, 2024. doi: 10.1162/tacl_a_00638.

[4] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, “LLMLingua: Compressing prompts for accelerated inference of large language models,” Proc. EMNLP, pp.13358–13376, Dec. 2023. doi: 10.18653/v1/2023.emnlp-main.825.

[5] Z. Pan, Q. Wu, H. Jiang, M. Xia, X. Luo, J. Zhang, Q. Lin, V. Rühle, Y. Yang, C.-Y. Lin, H. V. Zhao, L. Qiu, and D. Zhang, “LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression,” Findings of ACL: ACL 2024, pp.963–981, Aug. 2024. doi: 10.18653/v1/2024.findings-acl.57.

[6] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang, and L. Qiu, “LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression,” Proc. ACL, vol.1, pp.1658–1677, Aug. 2024. doi: 10.18653/v1/2024.acl-long.91.

[7] M. Kang, W.-N. Chen, D. Han, H. A. Inan, L. Wutschitz, Y. Chen, R. Sim, and S. Rajmohan, “ACON: Optimizing context compression for long-horizon LLM agents,” Proc. ICML, PMLR, vol.306, pp.55321–55375, July 2026.

[8] B. Xu, Z. Xue, D. Chen, C. Fu, C. Wu, C. Huang, C. Jiang, J. Fang, X. Deng, Y. Chen, Y. Yao, X. Wang, J. Shang, H. Chen, and N. Zhang, “TokenPilot: Cache-efficient context management for LLM agents,” Findings of ACL: EMNLP 2026, pp.19034–19049, Oct. 2026.

[9] J. S. Park, J. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” Proc. ACM UIST, article 2, pp.1–22, Oct. 2023. doi: 10.1145/3586183.3606763.

[10] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” Advances in Neural Information Processing Systems, vol.36, pp.8634–8652, 2023. doi: 10.52202/075280-0377.

[11] Q. Lu, L. Ding, S. Cao, X. Liu, K. Zhang, J. Zhang, and D. Tao, “Runaway is ashamed, but helpful: On the early-exit behavior of large language model-based agents in embodied environments,” Findings of ACL: EMNLP 2025, pp.24014–24027, Nov. 2025. doi: 10.18653/v1/2025.findings-emnlp.1304.

[12] H. Trivedi, T. Khot, M. Hartmann, R. Manku, V. Dong, E. Li, S. Gupta, A. Sabharwal, and N. Balasubramanian, “AppWorld: A controllable world of apps and people for benchmarking interactive coding agents,” Proc. ACL, vol.1, pp.16022–16076, Aug. 2024. doi: 10.18653/v1/2024.acl-long.850.

Abstract
We independently evaluated history summarization methods and termination control based on API execution records for LLM agents. Counting all model calls, including dedicated summarization, showed reductions in total tokens while preserving success on known tasks under the evaluated conditions.
Key words: LLM agents, harness, history summarization, termination control, token usage